

CSCI89 Lecture 4: 텍스트 표현 및 합성곱 신경망

UIUC Student

개요

요약

본 문서는 CSCI89 Lecture 4 강의 내용을 바탕으로, 자연어 처리(NLP)의 핵심 개념인 텍스트 전처리, 텍스트를 숫자로 표현하는 다양한 기법(Bag-of-Words, N-grams, Word Embedding), 그리고 이미지 처리뿐만 아니라 NLP에도 활용되는 합성곱 신경망(CNN)의 원리와 적용 방안을 상세히 설명합니다. 특히, 희소 표현의 문제점과 이를 해결하기 위한 임베딩의 중요성, 그리고 CNN의 가중치 공유 및 지역적 연결 특성을 강조하여 비전공자도 쉽게 이해할 수 있도록 구성되었습니다. 과제 피드백과 실제 구현 예시를 통해 이론과 실습의 간극을 줄이고, 시험 대비를 위한 핵심 내용을 체계적으로 정리합니다.

용어 정리

다음 표는 본 문서에서 사용되는 주요 용어들을 쉽게 설명하고 원어 및 비고를 함께 제시합니다.

용어	쉬운 설명	원어	비고
토큰화	텍스트를 의미 있는 작은 단위(단어, 문자 등)로 나누는 과정	Tokenization	NLP의 첫 단계
스테밍	단어의 어미나 접사를 제거하여 어간을 추출하는 과정	Stemming	실제 단어가 아닐 수 있음
표제어 추출	단어를 사전형(기본형)으로 변환하는 과정	Lemmatization	품사 정보 활용, 스테밍보다 정확
단어 가방	문서에서 단어의 출현 빈도만으로 텍스트를 표현하는 방식	Bag-of-Words (BoW)	단어 순서 정보 손실
N-그램	N개의 연속된 단어를 하나의 토큰으로 간주하는 방식	N-grams	단어 간의 지역적 문맥 포착
단어 임베딩	단어를 저차원의 밀집 벡터로 표현하여 의미를 반영하는 방식	Word Embedding	희소성 문제 해결, 의미 유사성 반영
희소 표현	대부분의 값이 0인 벡터나 행렬로 데이터를 표현하는 방식	Sparse Representation	BoW의 주요 단점
어휘 집합	텍스트 데이터에서 추출된 고유한 단어들의 집합	Vocabulary	모델 학습의 기준
Oov	어휘 집합에 없는 단어	Out-of-Vocabulary	새로운 단어 처리 방식
합성곱 신경망	이미지나 시퀀스 데이터의 지역적 특징을 추출하는 신경망	Convolutional Neural Network (CNN)	가중치 공유, 지역적 연결
필터/커널	합성곱 연산에 사용되는 작은 행렬	Filter / Kernel	특징 추출기 역할
패딩	합성곱 연산 시 이미지 경계에 0을 추가하는 기법	Padding	출력 크기 조절
스트라이드	필터가 이동하는 간격	Stride	특징 맵의 크기 조절
풀링	특징 맵의 크기를 줄이고 중요한 특징을 강조하는 기법	Pooling	Max Pooling, Average Pooling
Flatten	다차원 데이터를 1차원 벡터로 변환하는 계층	Flatten	완전 연결 계층으로의 입력 준비

1 과제 2 피드백 및 논의

이번 강의는 지난 과제 2에 대한 피드백과 함께 시작되었습니다. 학생들은 과제를 수행하면서 겪었던 기술적인 문제와 개념적인 어려움에 대해 질문하고, 강사는 이에 대한 설명을 제공했습니다.

1.1 기술적 문제 및 해결

- **TensorFlow 설치 문제:** 한 학생은 로컬 TensorFlow 설치 문제가 아님을 확인했음에도 불구하고 기술적인 문제로 과제를 완료하지 못했다고 언급했습니다. 다른 노트북에서는 신경망이 정상적으로 작동했기 때문에, 문제는 로컬 환경이 아닌 다른 곳에 있었을 가능성이 높습니다.
- **커널 충돌:** 과제의 첫 번째 부분이 계속해서 커널 충돌을 일으켰다고 합니다. 이는 코드나 환경 설정의 특정 부분에서 발생할 수 있는 문제입니다.
- **정확도 75% 미달:** 과제의 두 번째, 세 번째 부분에서는 순환 신경망(Recurrent Network)을 사용하여 정확도를 75% 이상으로 올리지 못했다고 합니다. 이는 모델 설계나 하이퍼파라미터 튜닝의 어려움을 시사합니다.

요약

강사는 앞으로는 과제에서 원-핫 인코딩(One-Hot Encoding) 방식을 직접 구현할 필요가 없을 것이라고 언급했습니다. 이는 원-핫 인코딩이 희소 표현(Sparse Representation) 문제를 야기하고 계산 비용이 높기 때문입니다. 대신, 임베딩(Embedding) 기법을 사용하여 텍스트를 효율적으로 표현하는 방법을 배울 것이라고 설명했습니다.

1.2 희소 표현(Sparse Representation)의 어려움

핵심 개념

희소 표현(Sparse Representation)은 대부분의 값이 0인 벡터나 행렬로 데이터를 표현하는 방식입니다. 원-핫 인코딩이 대표적인 예시이며, 이는 저장 공간 낭비와 높은 계산 비용을 초래합니다.

강사는 원-핫 인코딩을 사용하면 벡터가 매우 길고 대부분 0으로 채워지는 '희소 표현'이 된다고 설명했습니다. 이는 다음과 같은 문제점을 야기합니다.

- **높은 차원:** 어휘 집합(Vocabulary)의 크기가 커질수록 벡터의 차원이 기하급수적으로 증가합니다.
- **저장 공간 낭비:** 대부분의 값이 0이므로, 실제 유용한 정보는 적은데도 많은 저장 공간을 차지합니다.

- **계산 비용 증가:** 신경망의 입력으로 사용될 때, 많은 수의 0을 처리해야 하므로 계산이 비효율적입니다.
- **의미 손실:** 단어 간의 의미적 관계를 전혀 반영하지 못합니다.

예시

예를 들어, 200개의 단어로 구성된 어휘 집합이 있다면, 각 단어는 200차원의 벡터로 표현됩니다. 이 중 단어가 나타나는 위치에만 1이 있고 나머지는 모두 0인 벡터가 됩니다. 이는 매우 비효율적입니다.

1.3 손실(Loss) 폭발 문제 (Sigmoid + Cross Entropy)

한 학생이 10번째 에포크(epoch)에서 손실(loss)이 비정상적으로 폭발하는 현상(ludicrous loss)에 대해 질문했습니다. 이는 딥러닝 모델 학습에서 흔히 발생하는 문제 중 하나입니다.

• 문제 발생 원인:

- 모델의 마지막 계층이 시그모이드(sigmoid) 함수이고, 손실 함수로 교차 엔트로피(cross entropy)를 사용했을 때 발생할 수 있습니다.
- 시그모이드 함수는 출력이 0 또는 1에 매우 가까워질 때, 모델이 특정 답에 대해 극도로 확신하지만 실제로는 틀린 경우(오탐 또는 오음성)에 문제가 됩니다.
- 이때 교차 엔트로피 손실 함수는 로그(log) 함수를 포함하는데, 로그 함수의 입력이 0에 매우 가까워지면 손실 값이 무한대로 발산하게 됩니다($\log(0) \rightarrow -\infty$). 예를 들어, 모델이 정답을 1로 확신했지만 실제 정답은 0인 경우, 손실 계산 시 $\log(0)$ 과 유사한 상황이 발생하여 손실이 폭발합니다.
- 특히, 단 하나의 데이터 포인트에서 모델이 잘못된 답에 대해 과도하게 확신할 때 이러한 현상이 발생할 수 있습니다.
- **흥미로운 관찰:** 손실이 폭발했음에도 불구하고, 해당 모델이 75% 이상의 정확도를 달성한 첫 번째 모델이었다는 점입니다. 이는 손실 함수가 특정 이상치(anomaly)에 매우 민감하게 반응할 수 있음을 보여줍니다.
- **해결책:** 학생은 드롭아웃(Dropout) 계층을 추가하여 이 문제를 해결했다고 보고했습니다. 드롭아웃은 과적합(overfitting)을 방지하고 모델의 일반화 성능을 향상시키는 데 도움이 되는 기법입니다.

주의

손실 폭발 문제는 모델이 특정 예측에 대해 과도하게 확신할 때 발생할 수 있습니다. 특히 시그모이드 출력과 교차 엔트로피 손실 함수를 함께 사용할 때 주의해야 합니다. 드롭아웃과 같은 정규화 기법이 해결책이 될 수 있습니다.

2 텍스트 전처리 및 표현 기법

자연어 처리(NLP)에서 텍스트 데이터를 모델이 이해할 수 있는 형태로 변환하는 것은 매우 중요합니다. 이를 위해 다양한 전처리 및 표현 기법이 사용됩니다.

2.1 토큰화(Tokenization)

핵심 개념

토큰화(Tokenization)는 텍스트를 의미 있는 작은 단위인 '토큰(token)'으로 분리하는 과정입니다. 이 토큰은 단어, 서브워드(subword), 또는 문자 단위가 될 수 있습니다.

2.1.1 정의 및 목적

토큰화의 주된 목적은 텍스트를 분석 가능한 단위로 나누는 것입니다. 어떤 단위로 나눌지는 특정 NLP 작업의 특성과 데이터셋에 따라 달라집니다.

- **단어(Words):** 텍스트 분류(text classification)와 같은 작업에서는 단어 단위의 토큰화가 흔히 사용됩니다. 특정 데이터셋에 모델을 적용할 때 효과적입니다.
- **문자(Characters):** 모델을 다른 데이터셋으로 전이 학습(transfer learning)시키거나, 어휘 집합(vocabulary)에 없는 단어(OOV) 문제를 처리할 때 문자 단위 토큰화가 유용할 수 있습니다. 모든 문자가 토큰이 되므로, 새로운 단어에 대해서도 유연하게 대처할 수 있습니다.
- **서브워드(Subwords):** 단어와 문자 토큰화의 장점을 결합한 방식으로, 자주 등장하는 단어는 그대로 사용하고 희귀한 단어는 더 작은 의미 단위(서브워드)로 분리합니다.

예시

퀴즈 질문: 토큰화(Tokenization)는 무엇인가? **정답:** 텍스트를 특정 하위 단위(subunits)로 분할하는 것입니다. (B)

오답 예시:

- 텍스트를 소문자로 변환하는 것 (대소문자 변환은 토큰화의 한 단계일 수 있지만, 토큰화 자체는 아님)
- 단어를 어근 형태로 줄이는 것 (스테밍 또는 표제어 추출)

2.1.2 NLTK를 이용한 토큰화 예시

NLTK(Natural Language Toolkit)는 파이썬에서 널리 사용되는 NLP 라이브러리입니다. NLTK의 `word_tokenize` 함수는 문장을 단어 단위로 토큰화합니다. 이때 단순히 공백이나 구두점을 기준으로 나누는 것이 아니라, 문맥을 고려하여 똑똑하게 토큰을 분리합니다.

예시

퀴즈 질문: NLTK의 `word_tokenize` 함수를 사용하여 "J. R. R. Tolkien was born in 1892." 문장을 토큰화하면 어떤 결과가 나오는가? 정답: ['J.', 'R.', 'R.', 'Tolkien', 'was', 'born', 'in', '1892', '.'] (C)

설명:

- NLTK는 'J.', 'R.'과 같이 이니셜 뒤에 오는 마침표를 단어의 일부로 인식하여 하나의 토큰으로 처리합니다. 단순히 마침표를 분리하지 않습니다.
- '1892'와 같은 숫자도 하나의 토큰으로 인식합니다.

2.2 스테밍(Stemming)과 표제어 추출(Lemmatization)

핵심 개념

스테밍(Stemming)은 단어의 접두사(prefix)와 접미사(suffix)를 제거하여 단어의 어간(root form)을 추출하는 과정입니다. 표제어 추출(Lemmatization)은 단어를 사전에서 찾을 수 있는 유효한 기본형(base or root form)으로 변환하는 과정입니다.

2.2.1 정의 및 목적

두 기법 모두 단어의 형태를 정규화하여 같은 의미를 가진 단어들을 하나로 묶는 데 사용됩니다. 예를 들어, "running", "runs", "ran"과 같은 단어들을 "run"이라는 하나의 형태로 만듭니다.

- 스테밍(Stemming):

- 정의: 단순히 규칙 기반으로 접두사/접미사를 잘라내는 방식입니다.
- 예시: "running" → "run", "octopus" → "octop"
- 장점: 구현이 간단하고 계산 비용이 매우 저렴합니다.
- 단점: 추출된 어간이 실제 사전에 없는 단어일 수 있습니다. (예: "octop"은 실제 단어가 아님)

- 표제어 추출(Lemmatization):

- 정의: 단어의 품사(part of speech)를 고려하여 사전에 있는 유효한 기본형으로 변환합니다.
- 예시: "running" → "run", "better" → "good"
- 장점: 항상 유효한 단어를 반환하므로 의미 보존에 더 유리합니다.
- 단점: 품사 태깅(POS tagging)과 같은 추가적인 분석이 필요하여 스테밍보다 계산 비용이 높습니다.

2.2.2 스테밍의 단점

예시

퀴즈 질문: 표제어 추출(Lemmatization)과 비교했을 때 스테밍(Stemming)의 단점은 무엇인가? **정답:** 실제 단어를 생성하지 않을 수 있습니다. (B)

설명: "octopus"를 스테밍하면 "octop"이 되는데, 이는 사전에 없는 단어입니다. 반면 표제어 추출은 "octopus"를 그대로 유지하거나, 문맥에 따라 다른 유효한 단어로 변환할 것입니다.

주의

스테밍은 텍스트 분류나 감성 분석과 같이 단어의 정확한 형태보다는 의미적 유사성이 중요한 작업에서는 유용할 수 있습니다. 그러나 텍스트 생성이나 번역과 같이 실제 단어를 사용해야 하는 작업에서는 표제어 추출이 더 적합합니다.

2.3 불용어(Stop Words) 처리

핵심 개념

불용어(Stop Words)는 "and", "the", "is"와 같이 문장에서 자주 등장하지만, 텍스트의 핵심 의미를 파악하는 데 크게 기여하지 않는 단어들을 말합니다.

2.3.1 정의 및 목적

불용어는 텍스트 데이터의 차원을 줄이고, 모델이 더 중요한 단어에 집중하도록 돕기 위해 제거될 수 있습니다.

- **차원 축소:** 불용어를 제거하면 어휘 집합의 크기가 줄어들어 모델의 계산 효율성이 향상됩니다.
- **노이즈 감소:** 의미 없는 단어들을 제거함으로써 모델이 텍스트의 핵심 내용을 더 잘 학습할 수 있습니다.
- **적용 여부:** 불용어 제거는 항상 필수는 아닙니다. 예를 들어, "not funny"와 같은 부정어 표현은 "not"이 불용어일지라도 문맥상 매우 중요하므로 제거하지 않는 것이 좋습니다. N-그램을 사용하면 이러한 문제를 어느 정도 해결할 수 있습니다.

예시

일반적으로 불용어 목록은 특정 패키지(예: NLTK)에서 제공하며, 이를 활용하여 텍스트에서 해당 단어들을 제거할 수 있습니다. 하지만 CountVectorizer와 같은 일부 도구는 기본적으로 불용어를 제거하지 않으므로, 필요하다면 명시적으로 처리해야 합니다.

3 텍스트를 숫자로 표현하는 방법

기계 학습 모델은 텍스트를 직접 처리할 수 없으므로, 텍스트를 숫자로 변환하는 과정이 필수적입니다. 여기서는 대표적인 텍스트 표현 기법들을 살펴봅니다.

3.1 단어 가방(Bag-of-Words, BoW)

핵심 개념

단어 가방(Bag-of-Words, BoW) 모델은 문서를 단어들의 순서나 문법을 무시하고, 단지 단어들의 출현 빈도(frequency)만을 기반으로 벡터로 표현하는 기법입니다. 마치 단어들을 가방에 넣고 흔들어 순서를 섞는 것과 같다고 하여 '단어 가방'이라고 불립니다.

3.1.1 정의 및 원리

BoW 모델은 각 문서를 고정된 길이의 벡터로 변환합니다. 이 벡터의 각 요소는 어휘 집합(Vocabulary) 내 특정 단어의 출현 빈도를 나타냅니다.

- **어휘 집합 생성:** 먼저 전체 훈련 데이터셋(corpus)에서 모든 고유한 단어들을 수집하여 어휘 집합을 만듭니다. 이 어휘 집합은 모델의 참조 사전 역할을 합니다.
- **벡터 표현:** 각 문서는 어휘 집합의 길이와 같은 차원의 벡터로 표현됩니다. 벡터의 각 위치에는 해당 위치에 해당하는 어휘 집합의 단어가 문서에 나타난 횟수(빈도)가 기록됩니다.
- **시간 순서 손실:** BoW의 가장 큰 특징이자 한계는 단어의 순서 정보가 완전히 무시된다는 점입니다. "영화가 재미있다"와 "재미있는 영화가 아니다"는 BoW 관점에서는 유사하게 표현될 수 있습니다.

3.1.2 장점 및 응용 분야

- **간단하고 강력함:** 구현이 비교적 간단하며, 특정 문제(예: 텍스트 분류)에서는 강력한 성능을 보여줍니다.
- **텍스트 분류(Text Classification):** 문서의 주제를 분류하거나 스팸 메일을 감지하는 등의 작업에 유용합니다.
- **감성 분석(Sentiment Analysis):** 영화 리뷰나 상품 평가와 같이 텍스트의 긍정/부정 감성을 판단하는 데 사용될 수 있습니다. 단어의 순서보다는 특정 감성 단어의 출현 여부나 빈도가 중요할 때 효과적입니다.

3.1.3 단점 및 한계

- **시간 순서 손실:** 단어의 순서나 문법적 구조를 무시하므로, 문맥(context) 정보를 잃게 됩니다. 이는 번역이나 텍스트 생성과 같이 순서가 중요한 작업에는 부적합합니다.

- **희소 표현(Sparse Representation):** 어휘 집합의 크기가 클수록 대부분의 문서 벡터는 많은 0을 포함하는 희소 벡터가 됩니다. 이는 저장 공간 낭비와 계산 비효율성을 초래합니다.
- **의미 손실:** 단어들을 독립적인 특징으로 간주하므로, 단어 간의 의미적 관계(예: 동의어, 유의어)를 포착하지 못합니다.
- **높은 차원:** 어휘 집합이 커지면 벡터의 차원이 매우 높아져 모델의 파라미터 수가 증가하고 학습이 어려워질 수 있습니다.

3.1.4 BoW 생성 단계

BoW를 생성하는 일반적인 단계는 다음과 같습니다.

1. **토큰화(Tokenization):** 문서를 단어 단위의 토큰으로 분리합니다.
2. **어휘 집합 생성(Vocabulary Creation):** 전체 코퍼스(corpus)에서 모든 고유한 토큰을 수집하여 어휘 집합을 만듭니다. 이때 중복된 단어는 한 번만 포함됩니다.
3. **빈도 계산(Frequency Counting):** 각 문서에 대해 어휘 집합의 각 단어가 몇 번 나타나는지 계산하여 벡터를 생성합니다.

예시

예시 문장: "Henry Ford introduced the Model T. The Model T was revolutionary."

1. **토큰화:** ['Henry', 'Ford', 'introduced', 'the', 'Model', 'T', '.', 'The', 'Model', 'T', 'was', 'revolutionary', '.']

2. **어휘 집합 생성:** (고유한 단어들을 출현 순서대로 정렬) ['Henry', 'Ford', 'introduced', 'the', 'Model', 'T', 'was', 'revolutionary', '.']

3. **빈도 계산:**

- 'Henry': 1회
- 'Ford': 1회
- 'introduced': 1회
- 'the': 1회
- 'Model': 2회
- 'T': 2회
- 'was': 1회
- 'revolutionary': 1회
- '.': 2회

이를 벡터로 표현하면 각 단어의 위치에 해당 빈도수가 들어갑니다.

3.1.5 BoW 구현 예시

scikit-learn의 CountVectorizer CountVectorizer는 BoW 모델을 쉽게 생성할 수 있도록 해 주는 도구입니다.

```

1 from sklearn.feature_extraction.text import CountVectorizer
2
3 corpus = [
4     "Henry Ford introduced the Model T.",
5     "Ford's Model T was revolutionary."
6 ]
7
8 # CountVectorizer 인스턴스 생성
9 vectorizer = CountVectorizer()
10
11 # 코퍼스에 fit_transform 적용하여 어휘집합 학습 및 BoW 벡터 생성
12 X = vectorizer.fit_transform(corpus)
13
14 # 어휘집합 확인
15 vocabulary = vectorizer.get_feature_names_out()
16 print("어휘 집합:", vocabulary)
17
18 # BoW 벡터 확인 함수 ( 행렬을 일반 배열로 변환 )
19 print("BoW 표현:\n", X.toarray())
20
21 # 새 문서에 대한 BoW 생성
22 new_document = ["The impact of Model T was huge."]
23 X_new = vectorizer.transform(new_document)
24 print("새 문서 BoW 표현:\n", X_new.toarray())

```

Listing 1: scikit-learn CountVectorizer를 이용한 BoW 생성

예시

출력 예시 및 설명: CountVectorizer는 자동으로 토큰화하고 어휘 집합을 생성합니다.

- 어휘 집합: ['ford', 'henry', 'impact', 'introduced', 'model', 'of', 'revolutionary', 't', 'the', 'was'] (알파벳 순으로 정렬됨)
- 첫 번째 문서 BoW: [1, 1, 0, 1, 1, 0, 0, 1, 1, 0] (각 단어의 빈도)
- 두 번째 문서 BoW: [1, 0, 0, 0, 1, 0, 1, 1, 0, 1]
- 새 문서 BoW: [0, 0, 1, 0, 1, 1, 0, 1, 1, 1]
 - 'impact', 'of', 'huge'와 같은 단어는 훈련 코퍼스에 없었으므로, 해당 위치의 값은 0이 됩니다. (OOV 처리)
 - 'Model', 'T', 'the', 'was' 등 기존 어휘에 있는 단어는 빈도수가 반영됩니다.

NLTK를 이용한 BoW NLTK는 CountVectorizer보다 더 수동적인 접근 방식을 제공하여, 토큰화, 어휘 집합 생성, 빈도 계산 과정을 직접 제어할 수 있습니다. 이는 더 많은 유연성을 제공합니다.

```

1 from nltk.tokenize import word_tokenize
2 import collections
3
4 corpus = [
5     "Henry Ford introduced the Model T.",
6     "Ford's Model T was revolutionary."
7 ]
8
9 all_tokens = []
10 tokenized_documents = []
11
12 # 모든문서토큰화및토큰리스트생성
13 for doc in corpus:
14     tokens = word_tokenize(doc.lower()) # 소문자변환후토큰화
15     tokenized_documents.append(tokens)
16     all_tokens.extend(tokens)
17
18 # 어휘집합생성중복 ( 제거및 OOV 토큰추가 )
19 vocabulary = sorted(list(set(all_tokens)))
20 vocabulary.append('OOV') # Out-Of-Vocabulary 토큰추가
21
22 def create_bag_of_words_nltk(tokens, vocab):
23     bow_vector = [0] * len(vocab)
24     oov_count = 0
25     for token in tokens:
26         try:
27             idx = vocab.index(token)
28             bow_vector[idx] += 1
29         except ValueError:
30             oov_count += 1 # 어휘집합에없는단어는로 OOV 카운트
31     bow_vector[vocab.index('OOV')] = oov_count
32     return bow_vector
33
34 print("어휘 집합:", vocabulary)
35
36 # 각문서에대한 BoW 생성
37 bow_vectors = []
38 for tokens in tokenized_documents:
39     bow_vectors.append(create_bag_of_words_nltk(tokens, vocabulary))
40 print("훈련 문서 BoW 표현:\n", bow_vectors)
41
42 # 새문서에대한 BoW 생성
43 new_document_text = "The impact of Model T was huge."
44 new_tokens = word_tokenize(new_document_text.lower())

```

```

45 new_bow_vector = create_bag_of_words_nltk(new_tokens, vocabulary)
46 print("새 문서 BoW 표현:\n", new_bow_vector)

```

Listing 2: NLTK를 이용한 BoW 생성

예시

설명:

- NLTK는 word_tokenize를 사용하여 텍스트를 토큰화합니다.
- 어휘 집합은 수동으로 생성되며, 'OOV' 토큰을 추가하여 어휘 집합에 없는 단어를 처리할 수 있습니다.
- create_bag_of_words_nltk 함수는 주어진 토큰 리스트와 어휘 집합을 바탕으로 BoW 벡터를 생성합니다.
- 새 문서에 대해 transform 메서드 없이 직접 함수를 호출하여 BoW를 생성합니다. 이때 어휘 집합에 없는 단어들은 'OOV' 카운트에 합산됩니다.

SpaCy를 이용한 BoW SpaCy는 NLTK와 유사하게 유연성을 제공하면서도, 더 빠르고 효율적인 처리를 위한 기능을 내장하고 있습니다.

```

1 import spacy
2 from collections import Counter
3
4 # SpaCy 모델로드예 ( : 영어소형모델 )
5 # python -m spacy download en_core_web_sm
6 nlp = spacy.load("en_core_web_sm")
7
8 corpus = [
9     "Henry Ford introduced the Model T.",
10    "Ford's Model T was revolutionary."
11 ]
12
13 all_tokens = []
14 tokenized_documents = []
15
16 for doc_text in corpus:
17     doc = nlp(doc_text.lower()) # SpaCy Doc 객체생성
18     tokens = [token.text for token in doc if not token.is_punct] # 구두점
19     # 제외
20     tokenized_documents.append(tokens)
21     all_tokens.extend(tokens)
22
23 vocabulary = sorted(list(set(all_tokens)))
24 vocabulary.append('OOV')

```

```

25 def create_bag_of_words_spacy(tokens, vocab):
26     bow_vector = [0] * len(vocab)
27     oov_count = 0
28     for token in tokens:
29         try:
30             idx = vocab.index(token)
31             bow_vector[idx] += 1
32         except ValueError:
33             oov_count += 1
34     bow_vector[vocab.index('OOV')] = oov_count
35     return bow_vector
36
37 print("어휘 집합:", vocabulary)
38
39 bow_vectors = []
40 for tokens in tokenized_documents:
41     bow_vectors.append(create_bag_of_words_spacy(tokens, vocabulary))
42 print("훈련 문서 BoW 표현:\n", bow_vectors)
43
44 new_document_text = "The impact of Model T was huge."
45 new_doc = nlp(new_document_text.lower())
46 new_tokens = [token.text for token in new_doc if not token.is_punct]
47 new_bow_vector = create_bag_of_words_spacy(new_tokens, vocabulary)
48 print("새 문서 BoW 표현:\n", new_bow_vector)

```

Listing 3: SpaCy를 이용한 BoW 생성

예시**설명:**

- SpaCy는 nlp 객체를 통해 텍스트를 처리하고, token.text를 사용하여 토큰을 추출합니다.
- NLTK와 유사하게 수동으로 어휘 집합을 만들고 BoW 벡터를 생성합니다.
- SpaCy는 품사 태깅(POS tagging)이나 개체명 인식(NER)과 같은 고급 NLP 기능을 기본적으로 제공하므로, 이를 활용하여 더 정교한 토큰화나 전처리를 수행할 수 있습니다.

3.2 N-그램(N-grams)

핵심 개념

N-그램(N-grams)은 텍스트에서 N개의 연속된 단어(또는 문자)를 하나의 토큰으로 간주하는 기법입니다. 이는 단어 가방(BoW) 모델의 단점인 단어 순서 정보 손실을 부분적으로 보완하여 지역적인 문맥(local context)을 포착하는 데 도움을 줍니다.

3.2.1 정의 및 목적

N-그램은 단어의 순서를 고려하여 특정 단어 시퀀스가 함께 나타나는 빈도를 측정합니다.

- **유니그램(Unigram, 1-gram)**: 단어 하나하나를 토큰으로 간주합니다. 이는 일반적인 BoW 모델과 동일합니다.
- **바이그램(Bigram, 2-gram)**: 두 개의 연속된 단어를 하나의 토큰으로 간주합니다.
- **트라이그램(Trigram, 3-gram)**: 세 개의 연속된 단어를 하나의 토큰으로 간주합니다.

예시

예시 문장: "This movie was not funny."

유니그램: ['This', 'movie', 'was', 'not', 'funny']

바이그램: ['This movie', 'movie was', 'was not', 'not funny']

트라이그램: ['This movie was', 'movie was not', 'was not funny']

문맥 포착의 중요성: 감성 분석에서 "funny"는 긍정적이지만, "not funny"는 부정적인 의미를 가집니다. 바이그램은 이러한 부정적인 문맥을 포착하는 데 유용합니다.

3.2.2 장점 및 단점

- **장점:**
 - **지역적 문맥 포착**: 인접한 단어들 간의 관계를 반영하여 BoW보다 풍부한 정보를 제공합니다.
 - **감성 분석 개선**: "not good", "very bad"와 같은 표현의 의미를 더 정확하게 파악할 수 있습니다.
- **단점:**
 - **어휘 집합 폭발**: N이 커질수록 가능한 N-그램의 수가 기하급수적으로 증가하여 어휘 집합의 크기가 매우 커집니다. 이는 희소성 문제를 더욱 심화시키고, 저장 공간 및 계산 비용을 증가시킵니다.
 - **장거리 의존성 한계**: N-그램은 인접한 N개의 단어만을 고려하므로, 멀리 떨어진 단어들 간의 관계(장거리 의존성)는 포착할 수 없습니다.

- **최적의 N 선택**: N의 크기를 결정하는 것이 중요합니다. 일반적으로 2-그램이나 3-그램이 가장 유용하며, 그 이상은 효율성이 떨어집니다.

3.2.3 N-그램 구현 예시

scikit-learn의 CountVectorizer CountVectorizer는 ngram_range 파라미터를 통해 N-그램을 쉽게 생성할 수 있습니다.

```

1 from sklearn.feature_extraction.text import CountVectorizer
2
3 corpus = [
4     "Henry Ford introduced the Model T.",
5     "Ford's Model T was revolutionary."
6 ]
7
8 # ngram_range=(2, 2)로 바이그램만사용
9 vectorizer = CountVectorizer(ngram_range=(2, 2))
10
11 X = vectorizer.fit_transform(corpus)
12
13 vocabulary = vectorizer.get_feature_names_out()
14 print("바이그램 어휘집합 :", vocabulary)
15 print("바이그램 BoW 표현:\n", X.toarray())
16
17 # 새문서에대한바이그램      BoW 생성
18 new_document = ["The impact of Model T was huge."]
19 X_new = vectorizer.transform(new_document)
20 print("새 문서바이그램      BoW 표현:\n", X_new.toarray())

```

Listing 4: scikit-learn CountVectorizer를 이용한 N-그램 생성

예시

출력 예시 및 설명:

- 바이그램 어휘 집합: ['ford introduced', 'introduced model', 'model revolutionary', 'model was', 't was', 'the model']
- 새 문서 BoW: 대부분의 값이 0이 됩니다. 훈련 코퍼스가 작기 때문에 새 문서에서 훈련된 바이그램이 거의 나타나지 않을 수 있습니다.

NLTK를 이용한 N-그램 NLTK는 ngrams 함수를 제공하여 N-그램을 생성합니다.

```

1 from nltk.tokenize import word_tokenize
2 from nltk.util import ngrams
3 import collections
4

```

```

5 corpus = [
6     "Henry Ford introduced the Model T.",
7     "Ford's Model T was revolutionary."
8 ]
9
10 all_ngrams = []
11 tokenized_documents = []
12
13 for doc in corpus:
14     tokens = word_tokenize(doc.lower())
15     doc_ngrams = list(ngrams(tokens, 2)) # 바이그램생성
16     tokenized_documents.append(doc_ngrams)
17     all_ngrams.extend(doc_ngrams)
18
19 vocabulary = sorted(list(set(all_ngrams)))
20 vocabulary.append(('OOV',)) # OOV 토큰은 튜플형태로추가
21
22 def create_ngram_bow_nltk(ngrams_list, vocab):
23     bow_vector = [0] * len(vocab)
24     oov_count = 0
25     for ngram_item in ngrams_list:
26         try:
27             idx = vocab.index(ngram_item)
28             bow_vector[idx] += 1
29         except ValueError:
30             oov_count += 1
31     bow_vector[vocab.index(('OOV',))] = oov_count
32     return bow_vector
33
34 print("바이그램 어휘집합 :", vocabulary)
35
36 ngram_bow_vectors = []
37 for doc_ngrams in tokenized_documents:
38     ngram_bow_vectors.append(create_ngram_bow_nltk(doc_ngrams,
39                                                     vocabulary))
40
41 print("훈련 문서바이그램 BoW 표현:\n", ngram_bow_vectors)
42
43 new_document_text = "The impact of Model T was huge."
44 new_tokens = word_tokenize(new_document_text.lower())
45 new_ngrams = list(ngrams(new_tokens, 2))
46 new_ngram_bow_vector = create_ngram_bow_nltk(new_ngrams, vocabulary)
47 print("새 문서바이그램 BoW 표현:\n", new_ngram_bow_vector)

```

Listing 5: NLTK를 이용한 N-그램 생성

SpaCy를 이용한 N-그램 SpaCy는 토큰화된 문서 객체에서 N-그램을 생성할 수 있습니다.

```

1 import spacy

```

```

2 from collections import Counter
3
4 nlp = spacy.load("en_core_web_sm")
5
6 corpus = [
7     "Henry Ford introduced the Model T.",
8     "Ford's Model T was revolutionary."
9 ]
10
11 all_ngrams = []
12 tokenized_documents = []
13
14 for doc_text in corpus:
15     doc = nlp(doc_text.lower())
16     # 의SpaCy 는ngrams 직접제공되지않으므로 , 수동으로생성
17     tokens = [token.text for token in doc if not token.is_punct]
18     doc_ngrams = [(tokens[i], tokens[i+1]) for i in range(len(tokens)-1)
19                  ] # 바이그램
20     tokenized_documents.append(doc_ngrams)
21     all_ngrams.extend(doc_ngrams)
22
23 vocabulary = sorted(list(set(all_ngrams)))
24 vocabulary.append(('OOV',))
25
26 def create_ngram_bow_spacy(ngrams_list, vocab):
27     bow_vector = [0] * len(vocab)
28     oov_count = 0
29     for ngram_item in ngrams_list:
30         try:
31             idx = vocab.index(ngram_item)
32             bow_vector[idx] += 1
33         except ValueError:
34             oov_count += 1
35     bow_vector[vocab.index(('OOV',))] = oov_count
36     return bow_vector
37
38 print("바이그램 어휘집합 :", vocabulary)
39
40 ngram_bow_vectors = []
41 for doc_ngrams in tokenized_documents:
42     ngram_bow_vectors.append(create_ngram_bow_spacy(doc_ngrams,
43                                                     vocabulary))
44
45 print("훈련 문서바이그램 BoW 표현:\n", ngram_bow_vectors)
46
47 new_document_text = "The impact of Model T was huge."
48 new_doc = nlp(new_document_text.lower())
49 new_tokens = [token.text for token in new_doc if not token.is_punct]

```

```

47 new_ngrams = [(new_tokens[i], new_tokens[i+1]) for i in range(len(
    new_tokens)-1)]
48 new_ngram_bow_vector = create_ngram_bow_spacy(new_ngrams, vocabulary)
49 print("새 문서바이그램 BoW 표현:\n", new_ngram_bow_vector)

```

Listing 6: SpaCy를 이용한 N-그램 생성

3.3 단어 임베딩(Word Embedding)

핵심 개념

단어 임베딩(Word Embedding)은 단어를 저차원의 밀집(dense) 벡터로 표현하는 기법입니다. 이는 단어 간의 의미적 유사성을 벡터 공간에 반영하여, 희소 표현의 문제점을 해결하고 모델이 단어의 문맥적 의미를 학습할 수 있도록 돕습니다.

3.3.1 정의 및 필요성

BoW나 원-핫 인코딩은 단어를 독립적인 존재로 취급하며, 벡터가 매우 희소하고 고차원이라는 단점이 있습니다. 임베딩은 이러한 문제를 해결하기 위해 등장했습니다.

- **희소성 문제 해결:** 원-핫 인코딩은 어휘 집합의 크기만큼 차원을 가지며 대부분 0으로 채워지지만, 임베딩은 훨씬 낮은 차원(예: 50차원, 100차원)의 밀집 벡터를 사용합니다.
- **의미 유사성 반영:** 임베딩 벡터 공간에서 의미적으로 유사한 단어들은 서로 가까운 위치에 배치됩니다. 예를 들어, "왕(king)"과 "여왕(queen)"의 벡터는 "남자(man)"와 "여자(woman)"의 벡터 관계와 유사한 패턴을 보일 수 있습니다.
- **문맥 학습:** 임베딩은 단어가 사용되는 문맥을 통해 학습되므로, 단어의 의미를 더 잘 포착할 수 있습니다.

3.3.2 원-핫 인코딩과의 비교

- **원-핫 인코딩:**
 - **표현:** 어휘 집합 크기만큼의 차원을 가지는 희소 벡터. 특정 단어 위치에만 1, 나머지는 0.
 - **정보:** 단어의 존재 여부만 표현. 단어 간의 관계나 의미는 반영하지 못함.
 - **문제점:** 고차원, 희소성, 계산 비효율성.
- **단어 임베딩:**
 - **표현:** 훨씬 낮은 차원(예: 32, 64, 128차원)의 밀집 벡터. 모든 요소가 0이 아닌 실수 값.
 - **정보:** 단어의 의미적, 문맥적 정보를 벡터 공간에 압축하여 표현.
 - **장점:** 차원 축소, 희소성 해결, 의미 유사성 반영, 효율적인 계산.

3.3.3 임베딩의 원리

임베딩은 고차원의 희소 벡터(원-핫 인코딩)를 저차원의 밀집 벡터로 매핑하는 함수라고 생각할 수 있습니다.

예시

고양이(Cat) 단어 예시:

- **원-핫 인코딩:** 어휘 집합에서 'Cat'이 두 번째 단어라면, $[0, 1, 0, \dots, 0]$ 과 같은 200차원(예시)의 희소 벡터로 표현됩니다. 이는 200차원 공간의 한 축에 있는 점과 같습니다.
- **임베딩:** 이 200차원 벡터를 3차원(예시)의 밀집 벡터로 매핑합니다. 예를 들어, $[0.7, 1.2, 0.2]$ 와 같은 벡터가 됩니다. 이 벡터는 3차원 공간의 한 점을 나타내며, 더 효율적이고 의미 있는 표현이 됩니다.

3.3.4 실제 구현 (인덱스 직접 사용)

신경망에서 임베딩 계층(Embedding Layer)을 사용할 때, 우리는 단어를 직접 원-핫 인코딩 벡터로 변환하여 입력하지 않습니다. 대신, 각 단어의 고유한 정수 인덱스(index)를 직접 입력으로 사용합니다.

- **인덱스 시퀀스:** 문장은 $[1, 2, 5, 7, 4, \dots]$ 와 같은 단어 인덱스들의 시퀀스로 표현됩니다.
- **임베딩 계층:** 임베딩 계층은 이 인덱스를 입력으로 받아, 내부적으로 학습된 임베딩 행렬에서 해당 인덱스에 해당하는 행(row)을 찾아 밀집 벡터로 출력합니다.
- **계산 효율성:** 이는 원-핫 인코딩 벡터를 명시적으로 생성하고 이를 선형 변환하는 것과 수학적으로 동등하지만, 훨씬 더 효율적인 계산 방식입니다. 임베딩 계층은 필요한 행만 선택하므로, 불필요한 0과의 곱셈 연산을 피할 수 있습니다.

예시

임베딩 계층의 파라미터 수: 어휘 집합의 크기가 200이고, 각 단어를 3차원 벡터로 임베딩하고 싶다고 가정해 봅시다.

- 임베딩 계층은 내부적으로 200×3 크기의 행렬을 가집니다. 이 행렬의 각 행은 어휘 집합의 한 단어에 대한 3차원 임베딩 벡터를 나타냅니다.
- 따라서 총 파라미터 수는 $200 \times 3 = 600$ 개가 됩니다.
- 단어 인덱스(예: 1)가 입력되면, 임베딩 계층은 이 행렬의 첫 번째 행을 찾아 출력합니다. 이는 원-핫 벡터 $[1, 0, \dots, 0]$ 에 200×3 행렬을 곱하는 것과 동일한 효과를 냅니다.

주의

임베딩은 모델 학습 과정에서 역전파(backpropagation)를 통해 최적화됩니다. 즉, 임베딩 벡터의 값들은 특정 작업(예: 분류)의 손실 함수를 최소화하는 방향으로 학습됩니다. 따라서 임베딩 자체를 독립적으로 해결하는 것이 아니라, 전체 신경망의 일부로서 학습됩니다.

4 합성곱 신경망(Convolutional Neural Networks, CNNs)

핵심 개념

합성곱 신경망(Convolutional Neural Networks, CNNs)은 주로 이미지 처리 분야에서 뛰어난 성능을 보이는 신경망 아키텍처입니다. 하지만 텍스트 데이터를 배열(sequence of vectors)로 간주하여 자연어 처리(NLP) 문제에도 성공적으로 적용될 수 있습니다.

4.1 CNN 개요 및 NLP 적용

4.1.1 이미지 처리에서 NLP로의 확장

CNN은 이미지의 지역적 특징(local features)을 효과적으로 추출하도록 설계되었습니다. 이러한 특성은 텍스트 데이터에도 적용될 수 있습니다.

- **이미지:** 픽셀들의 2차원 배열로, 인접한 픽셀들 간에 강한 상관관계(correlation)가 존재합니다. CNN은 필터(filter)를 사용하여 이러한 지역적 패턴(예: 모서리, 질감)을 감지합니다.
- **텍스트:** 단어 임베딩을 통해 각 단어를 벡터로 표현하면, 문장은 벡터들의 시퀀스(sequence of vectors)가 됩니다. 이 시퀀스를 1차원 배열처럼 간주할 수 있습니다.
- **NLP 적용:** 텍스트에서 인접한 단어들(N-그램과 유사)은 문맥적 의미를 형성합니다. CNN은 필터를 사용하여 이러한 지역적 단어 패턴(예: 구문, 특정 표현)을 추출할 수 있습니다. 광학 문자 인식(OCR)도 이미지 처리의 일종으로 CNN이 활용됩니다.

예시

텍스트를 벡터 시퀀스 $[X_1, X_2, X_3, \dots, X_T]$ 로 표현할 때, 각 X_i 는 단어 임베딩 벡터입니다. 이 벡터 시퀀스는 마치 1차원 이미지처럼 간주될 수 있으며, CNN 필터는 이 시퀀스 위를 슬라이딩하며 지역적 특징을 추출합니다.

4.2 합성곱(Convolution) 연산

핵심 개념

합성곱(Convolution)은 입력 데이터(예: 이미지)와 필터(filter) 또는 커널(kernel)이라는 작은 행렬을 사용하여 새로운 특징 맵(feature map)을 생성하는 연산입니다. 필터는 입력 데이터 위를 슬라이딩하며 지역적 패턴을 감지합니다.

4.2.1 정의 및 직관적 이해

합성곱 연산은 신호 처리에서 유래한 개념입니다. 물리적인 센서가 특정 지점의 신호를 정확히 측정하기 어렵고, 항상 주변의 신호를 통합하여 측정하는 것과 유사합니다.

- **신호 처리 비유:** 어떤 슬릿(slit)을 통해 들어오는 빛의 강도를 측정할 때, 슬릿의 크기 때문에 특정 한 점의 강도만 측정할 수 없습니다. 슬릿을 통과하는 빛의 누적된(cumulative) 강도를 측정하게 됩니다. 이때 슬릿의 특성(필터)과 실제 신호(입력)가 겹치는 영역의 합이 측정값(출력)이 됩니다.
- **신경망에서의 합성곱:** 입력 데이터의 지역적인 부분에 필터의 가중치(weights)를 곱하고 모두 더하는 연산입니다. 이는 마치 작은 돋보기(필터)를 들고 이미지의 각 부분을 살펴보며 특징을 추출하는 것과 같습니다.

4.2.2 완전 연결 신경망(FCNN)과의 비교

CNN은 완전 연결 신경망(Fully Connected Neural Network, FCNN)과 비교하여 두 가지 중요한 특징을 가집니다.

특징	완전 연결 신경망 (FCNN)	합성곱 신경망 (CNN)
연결 방식	모든 입력 뉴런이 다음 계층의 모든 뉴런과 연결 (각 뉴런이 전체 입력 신호를 받음)	지역적 연결 (Local Connectivity) (각 뉴런이 입력의 특정 지역만 연결)
가중치 공유	각 뉴런이 고유한 가중치 집합을 가짐 (파라미터 수가 매우 많음)	가중치 공유 (Weight Sharing) (동일한 필터 가중치를 입력의 여러 위치에 적용)
파라미터 수	매우 많음	훨씬 적음
특징 감지	전역적 패턴 감지	지역적 패턴 감지 (위치 불변성)

- **지역적 연결(Local Connectivity):** CNN의 뉴런은 입력 데이터의 전체가 아닌 특정 지역(receptive field)에만 연결됩니다. 이는 이미지에서 특정 픽셀이 주변 픽셀과 더 강한 상관관계를 가지는 것과 유사합니다.
- **가중치 공유(Weight Sharing):** 동일한 필터(가중치 집합)가 입력 데이터의 모든 지역에 걸쳐 슬라이딩하며 적용됩니다.
 - **파라미터 감소:** FCNN은 모든 연결에 대해 고유한 가중치를 가지지만, CNN은 필터의 가중치만 학습하면 되므로 파라미터 수가 획기적으로 줄어듭니다.
 - **위치 불변성(Translation Invariance):** 이미지의 특정 특징(예: 고양이의 눈)이 어디에 나타나든 동일한 필터로 감지할 수 있습니다. 이는 "어떤 위치에 있든 같은 눈"이라는 직관을 반영합니다.

4.2.3 필터(Filter) / 커널(Kernel)

필터는 합성곱 연산의 핵심입니다. 이는 입력 데이터에서 특정 특징(예: 수직선, 수평선, 모서리)을 추출하는 역할을 하는 작은 행렬입니다. 필터의 각 요소는 학습 가능한 가중치입니다.

4.2.4 바이어스(Bias)

각 필터는 합성곱 연산 결과에 더해지는 바이어스(bias) 값을 가집니다.

4.2.5 활성화 함수(Activation Function)

합성곱 연산 결과에 비선형성을 추가하기 위해 활성화 함수(예: ReLU)가 적용됩니다.

4.2.6 패딩(Padding)

합성곱 연산 시 입력 데이터의 경계에 0을 추가하는 기법입니다.

- **Valid Padding:** 패딩을 사용하지 않는 경우입니다. 필터가 입력 데이터의 경계를 넘어가지 않으므로, 출력 특징 맵의 크기가 입력보다 작아집니다. (예: $N \times N$ 입력에 $F \times F$ 필터를 적용하면 $(N - F + 1) \times (N - F + 1)$ 크기의 출력이 나옴)
- **Same Padding:** 입력 데이터의 크기와 동일한 크기의 특징 맵을 출력하기 위해 경계에 0을 추가합니다.

4.2.7 스트라이드(Stride)

필터가 입력 데이터 위를 이동하는 간격입니다.

- **Stride = 1:** 필터가 한 칸씩 이동합니다.
- **Stride > 1:** 필터가 여러 칸씩 건너뛰며 이동합니다. 이는 특징 맵의 크기를 줄이는 효과가 있습니다.

주의

입력 데이터의 차원을 보존하려면 패딩은 'Same'으로, 스트라이드는 '1'로 설정해야 합니다. 그렇지 않으면 특징 맵의 크기가 줄어듭니다.

4.2.8 합성곱 연산 예시 (2D 배열)

다음은 2D 배열에 필터를 적용하는 합성곱 연산의 예시입니다.

예시

입력 배열 (Input Array):

1	2	3
4	5	6
7	8	9

필터 (Filter):

1	1
2	2

바이어스 (Bias): 100

연산 과정:

1. 필터를 입력 배열의 좌측 상단에 배치합니다 (1, 2, 4, 5).
2. 필터의 각 요소와 겹치는 입력 배열의 요소를 곱하고 모두 더합니다: $(1 \times 1) + (1 \times 2) + (2 \times 4) + (2 \times 5) = 1 + 2 + 8 + 10 = 21$.
3. 여기에 바이어스를 더합니다: $21 + 100 = 121$.
4. 활성화 함수(예: ReLU)를 적용합니다. 만약 $\text{ReLU}(\max(z, 0))$ 를 사용하면, 121이 출력됩니다.
5. 필터를 스트라이드(기본값 1)만큼 이동하여 다음 위치에 대해 동일한 연산을 반복합니다.

출력 특징 맵 (Output Feature Map): (Valid Padding, Stride=1 가정)

121	128
135	142

4.3 풀링(Pooling) 계층

핵심 개념

풀링(Pooling) 계층은 합성곱 계층의 출력인 특징 맵의 크기를 줄여(다운샘플링) 모델의 파라미터 수를 감소시키고, 과적합을 방지하며, 특징의 위치 변화에 대한 강건성(robustness)을 높이는 역할을 합니다.

4.3.1 목적

- 차원 축소: 특징 맵의 공간적 크기를 줄여 다음 계층의 계산량을 감소시킵니다.
- 과적합 방지: 모델의 복잡도를 줄여 일반화 성능을 향상시킵니다.
- 위치 불변성 강화: 특징의 미세한 위치 변화에도 동일한 특징으로 인식하도록 돕습니다.

4.3.2 Max Pooling

가장 널리 사용되는 풀링 기법입니다.

- **원리:** 특징 맵의 특정 지역(예: 2×2 영역)에서 가장 큰 값(최대값)을 추출하여 해당 지역을 대표하는 값으로 사용합니다.
- **장점:** 가장 두드러진 특징을 보존하고, 이미지의 흐릿함(blur)을 줄이는 데 효과적입니다. 평균 풀링(Average Pooling)은 특징 맵을 흐릿하게 만들 수 있습니다.

예시

Max Pooling 예시 (2×2 필터, Stride=2): 입력 특징 맵:

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Max Pooling 적용:

- 좌측 상단 2×2 영역 [1, 2, 5, 6]에서 최대값은 6.
- 우측 상단 2×2 영역 [3, 4, 7, 8]에서 최대값은 8.
- 좌측 하단 2×2 영역 [9, 10, 13, 14]에서 최대값은 14.
- 우측 하단 2×2 영역 [11, 12, 15, 16]에서 최대값은 16.

출력 특징 맵:

6	8
14	16

 입력 크기가 4×4 에서 2×2 로 줄어들었습니다.

4.4 CNN 아키텍처 예시 및 Flatten

CNN 모델은 여러 합성곱 계층과 풀링 계층을 쌓아 올린 후, 최종적으로 완전 연결 계층을 통해 분류나 회귀와 같은 작업을 수행합니다.

예시

CNN 모델 아키텍처 예시:

- **입력 계층 (Input Layer):** Input Shape: (32, 32, 3)
 - 32 × 32 픽셀 크기의 이미지.
 - 3개의 채널(예: Red, Green, Blue)을 가집니다.
- **합성곱 계층 1 (Conv2D Layer 1):** Conv2D(32, (3, 3))
 - 32개의 필터(feature map)를 사용합니다.
 - 각 필터의 크기는 3 × 3입니다.
 - 입력 채널이 3개이므로, 각 필터는 3 × 3 × 3의 가중치를 가집니다.
 - 패딩이 'valid'이고 스트라이드가 1인 경우, 출력 크기는 $(32 - 3 + 1) \times (32 - 3 + 1) = 30 \times 30$ 이 됩니다.
 - 출력: (30, 30, 32) (높이, 너비, 채널 수)
- **맥스 풀링 계층 1 (MaxPooling2D Layer 1):** MaxPooling2D((2, 2))
 - 2 × 2 크기의 풀링 필터를 사용합니다.
 - 각 특징 맵의 크기를 절반으로 줄입니다.
 - 출력: (15, 15, 32) (채널 수는 유지)
- **합성곱 계층 2 (Conv2D Layer 2):** Conv2D(64, (3, 3))
 - 64개의 필터를 사용합니다.
 - 각 필터의 크기는 3 × 3입니다.
 - 입력 채널이 32개이므로, 각 필터는 3 × 3 × 32의 가중치를 가집니다.
 - 출력 크기는 $(15 - 3 + 1) \times (15 - 3 + 1) = 13 \times 13$ 이 됩니다.
 - 출력: (13, 13, 64)
- **맥스 풀링 계층 2 (MaxPooling2D Layer 2):** MaxPooling2D((2, 2))
 - 출력: (6, 6, 64) (13을 2로 나누면 6.5이므로, 반올림 또는 버림에 따라 6이 됨)
- **Flatten 계층:**
 - 다차원 특징 맵을 1차원 벡터로 평탄화(reshape)합니다.
 - (6, 6, 64) → $6 \times 6 \times 64 = 2304$ 차원의 벡터.
 - 이 계층은 파라미터가 없으며, 단순히 데이터의 형태를 변경합니다.
- **완전 연결 계층 (Dense Layer):** Dense(10, activation='softmax')
 - 10개의 출력 뉴런을 가집니다 (예: 10가지 클래스 분류).
 - 소프트맥스(softmax) 활성화 함수를 사용하여 각 클래스에 대한 확률을 출력합니다.

4.4.1 Flatten 계층

핵심 개념

Flatten 계층은 합성곱 계층이나 풀링 계층에서 생성된 다차원의 특징 맵을 완전 연결 계층 (Dense Layer)의 입력으로 사용하기 위해 1차원 벡터로 변환하는 역할을 합니다.

- **목적:** CNN의 특징 추출 부분(합성곱 + 풀링)과 분류/회귀 부분(완전 연결 계층)을 연결합니다.
- **비용:** Flatten 계층 자체는 학습 가능한 파라미터가 없으므로, 계산 비용이 발생하지 않습니다. 단순히 데이터의 형태를 재구성하는 작업입니다.

4.4.2 실제 이미지 데이터셋(MNIST)에서의 CNN vs. Flattening 비교

MNIST와 같이 깔끔하게 중앙에 정렬된 숫자 이미지 데이터셋에서는 이미지를 바로 Flatten하여 완전 연결 신경망에 입력해도 좋은 성능을 얻을 수 있습니다.

주의

현실 세계 데이터의 문제점: 현실 세계의 이미지(예: 고양이 사진)는 숫자가 항상 중앙에 있거나 크기가 일정하지 않습니다. 고양이가 이미지의 왼쪽 상단에 있을 수도 있고, 오른쪽 하단에 있을 수도 있으며, 크기도 다양합니다.

- 만약 이미지를 바로 Flatten하여 FCNN에 입력하면, 모델은 특정 위치에서만 특징을 인식하도록 학습될 수 있습니다.
- 고양이가 다른 위치로 이동하면, 이전에 학습된 뉴런들은 해당 특징을 인식하지 못하게 되어 성능이 저하됩니다.

CNN의 장점: CNN의 가중치 공유와 지역적 연결 특성은 이러한 위치 불변성 문제를 해결합니다. 필터는 이미지의 어느 위치에 있든 동일한 특징(예: 고양이의 귀)을 감지할 수 있으므로, 현실 세계의 다양한 이미지에 더 강건하게 작동합니다.

4.4.3 1x1 필터 (Inception Module)

핵심 개념

1x1 필터는 합성곱 계층에서 1×1 크기의 필터를 사용하는 것을 말합니다. 이는 특징 맵의 공간적 해상도를 유지하면서 채널(깊이) 차원을 조절하는 데 주로 사용됩니다.

- **목적:**

- 채널 차원 축소/확대: 1×1 필터는 각 픽셀에 대해 채널 방향으로 선형 변환을 수행합니다. 이를 통해 특징 맵의 채널 수를 줄이거나 늘릴 수 있습니다. 이는 계산 비용을 줄이는 데 매우 효과적입니다.
- 비선형성 추가: 1×1 합성곱 뒤에 활성화 함수를 적용하여 비선형성을 추가할 수 있습니다.
- **Inception Module:** GoogleNet(Inception)과 같은 고급 CNN 아키텍처에서 널리 사용됩니다. Inception 모듈은 1×1 , 3×3 , 5×5 필터와 풀링을 병렬로 적용하여 다양한 스케일의 특징을 추출하며, 이때 1×1 필터는 주로 채널 차원을 줄여 계산 비용을 제어하는 데 사용됩니다.

5 학습 로드맵 및 시험 대비 체크리스트

5.1 학습 로드맵

본 강의 내용을 효과적으로 학습하기 위한 권장 로드맵입니다.

1. **기초 다지기:** 텍스트 전처리(토큰화, 스테밍, 표제어 추출, 불용어)의 정의와 목적을 명확히 이해합니다.
2. **핵심 개념 이해:** 단어 가방(BoW)과 N-그램의 원리, 장단점, 그리고 희소 표현의 문제점을 파악합니다.
3. **문제 해결 방안:** 단어 임베딩이 희소 표현 문제를 어떻게 해결하고 단어의 의미를 어떻게 반영하는지 깊이 이해합니다.
4. **새로운 아키텍처:** 합성곱 신경망(CNN)의 기본 원리(합성곱, 필터, 풀링, 가중치 공유, 지역적 연결)를 학습하고, NLP에 어떻게 적용되는지 파악합니다.
5. **실습 및 코드:** 제공된 코드 예시(scikit-learn, NLTK, SpaCy)를 직접 실행하고, 각 도구의 사용법과 유연성 차이를 경험합니다.
6. **개념 비교:** BoW vs. 임베딩, 스테밍 vs. 표제어 추출, FCNN vs. CNN의 차이점을 명확히 구분합니다.

5.2 시험 대비 체크리스트

No.	항목	완료 여부
1	토큰화의 정의와 다양한 토큰 단위(단어, 문자, 서브워드)를 설명할 수 있는가?	☐
2	스테밍과 표제어 추출의 차이점, 장단점, 그리고 적용 시기를 설명할 수 있는가?	☐
3	단어 가방(BoW) 모델의 원리, 장단점, 그리고 희소 표현 문제를 설명할 수 있는가?	☐
4	N-그램의 정의와 지역적 문맥 포착 원리를 설명하고, N-그램의 장단점을 열거할 수 있는가?	☐
5	단어 임베딩이 희소 표현 문제를 어떻게 해결하고, 단어의 의미를 어떻게 반영하는지 설명할 수 있는가?	☐
6	임베딩 계층의 파라미터 수를 계산할 수 있는가?	☐
7	합성곱 신경망(CNN)이 완전 연결 신경망(FCNN)과 다른 두 가지 주요 특징(지역적 연결, 가중치 공유)을 설명할 수 있는가?	☐
8	합성곱 연산의 필터, 패딩(Valid/Same), 스트라이드의 역할을 설명할 수 있는가?	☐
9	맥스 풀링(Max Pooling)의 목적과 작동 방식을 설명할 수 있는가?	☐
10	CNN 아키텍처 예시(Conv2D, MaxPooling2D, Flatten, Dense)를 보고 각 계층의 입출력 형태와 파라미터 변화를 이해할 수 있는가?	☐
11	Flatten 계층의 역할과 파라미터 수를 설명할 수 있는가?	☐
12	1x1 필터의 역할과 Inception Module에서의 활용을 설명할 수 있는가?	☐
13	scikit-learn, NLTK, SpaCy를 이용한 BoW 및 N-그램 구현 코드를 이해하고 수정할 수 있는가?	☐

6 FAQ

- **Q:** 단어 가방(BoW)에서 단어의 빈도를 사용하지 않고 단순히 단어의 존재 여부(0 또는 1)만 사용해도 되나요? **A:** 네, 가능합니다. 단어의 존재 여부만 사용하는 것은 '이진 단어 가방(Binary Bag-of-Words)'이라고 불리며, 계산 비용을 줄일 수 있습니다. 하지만 "great"과 같은 긍정적인 단어가 여러 번 반복될 때의 강한 긍정적 감성을 포착하지 못할 수 있습니다. 빈도를 사용하면 이러한 정보를 더 잘 반영할 수 있습니다.
- **Q:** N-그램을 사용할 때 N의 크기를 어떻게 결정해야 하나요? **A:** 일반적으로 2-그램(바이그램)이나 3-그램(트라이그램)이 가장 유용합니다. N이 너무 커지면 어휘 집합이 폭발적으로 증가하여 희소성 문제가 심화되고 계산 비용이 매우 높아집니다. 감성 분석과 같은 작업에서는 2-그램과 3-그램을 함께 사용하는 것이 최상의 성능을 낼 수 있습니다.
- **Q:** 임베딩 계층에서 입력으로 원-핫 인코딩 벡터 대신 정수 인덱스를 직접 사용하는 이유는 무엇인가요? **A:** 계산 효율성 때문입니다. 원-핫 인코딩 벡터를 명시적으로 생성하여 입력하는 것은 많은 0과의 불필요한 곱셈 연산을 유발합니다. 정수 인덱스를 직접 사용하면, 임베딩 계층이 내부적으로 학습된 임베딩 행렬에서 해당 인덱스에 해당하는 행(단어 벡터)을 바로 선택하여 출력할 수 있어 훨씬 빠르고 효율적입니다.
- **Q:** 과제 채점은 언제 이루어지나요? **A:** 과제 채점은 일관성을 위해 모든 학생의 과제를 채점한 후 일괄적으로 공개됩니다. 여러 채점자가 있을 수 있으므로, 모든 채점 결과를 확인하고 일관성을 유지하기 위해 시간이 소요됩니다.
- **Q:** 불용어(Stop Words)는 항상 제거해야 하나요? **A:** 아닙니다. 불용어 제거는 텍스트의 차원을 줄이고 노이즈를 감소시키는 데 도움이 되지만, 때로는 문맥상 중요한 정보를 제거할 수도 있습니다. 예를 들어, "not good"과 같은 부정 표현에서 "not"을 제거하면 의미가 완전히 달라집니다. 따라서 작업의 특성과 데이터셋을 고려하여 불용어 제거 여부를 결정해야 합니다. N-그램을 사용하면 불용어가 포함된 구문도 하나의 토큰으로 처리하여 문맥을 보존할 수 있습니다.

7 빠르게 훑어보기 (1 페이지 요약)

CSCI89 Lecture 4 핵심 요약

과제 피드백: 원-핫 인코딩의 희소성 문제, 손실 폭발(Sigmoid+Cross Entropy) 및 Dropout 해결.

텍스트 전처리:

- **토큰화:** 텍스트를 단어/문자 단위로 분리. NLTK/SpaCy 활용.
- **스테밍 vs. 표제어 추출:** 어간 추출 vs. 사전형 변환. 정확도와 비용의 트레이드 오프.
- **불용어:** 의미 없는 단어 제거로 차원 축소. 문맥 고려하여 선택적 적용.

텍스트 표현 기법:

- **단어 가방(BoW):** 단어 빈도 기반. 순서 정보 손실, 희소성 문제. 텍스트 분류에 유용.
- **N-그램:** N개 연속 단어 토큰화. 지역적 문맥 포착. 어휘 집합 폭발 주의.
- **단어 임베딩:** 단어를 저차원 밀집 벡터로 표현. 희소성 해결, 의미 유사성 반영. 인덱스 직접 입력으로 효율성 증대.

합성곱 신경망(CNN):

- **NLP 적용:** 텍스트를 벡터 시퀀스(1D 배열)로 간주하여 지역적 패턴 추출.
- **합성곱 연산:** 필터(가중치)가 입력 위를 슬라이딩하며 특징 추출.
- **FCNN vs. CNN:** CNN은 지역적 연결과 가중치 공유로 파라미터 감소 및 위치 불변성 확보.
- **필터/패딩/스트라이드:** 특징 추출 및 특징 맵 크기 조절.
- **풀링 계층:** 특징 맵 크기 축소(Max Pooling), 과적합 방지.
- **Flatten 계층:** 다차원 특징 맵을 1차원 벡터로 변환하여 완전 연결 계층에 연결. 파라미터 없음.
- **1x1 필터:** 채널 차원 조절, 계산 효율성 증대.